

# ByteFormer 微调 MNIST

## 零基础图像分类实验课

按步骤运行 • 查看真实结果 • 亲手修改一个参数

<https://github.com/Franklin-L/byteformer-mnist-course>

# 01 任务要求与完成路线

---

## ■ 完成一次真实微调

加载官方预训练 ByteFormer，识别 MNIST 手写数字 0—9；教师正式测试 97.07%。

## ■ 亲手完成三个操作

自行设置轮数并微调 → 观察验证曲线 → 查看并解释一个错例。

## ■ 交付自己的实验记录

保留指标、曲线、错例和 Word 报告，不能用教师结果代替自己的运行。

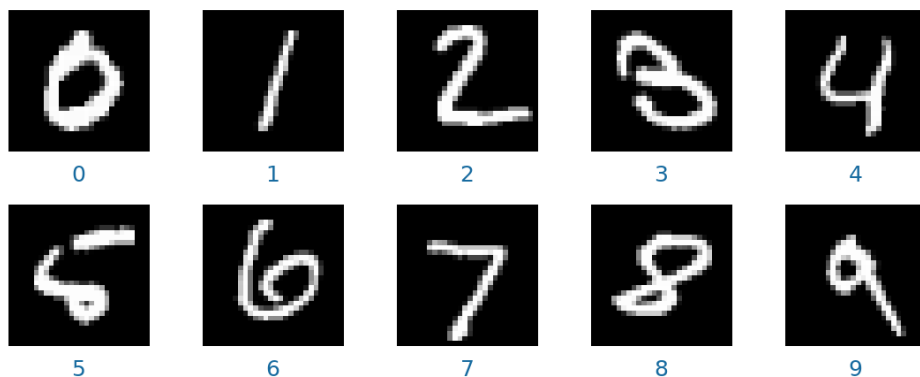
## 02 实验原理：文件字节输入 + 预训练微调

手写数字图 → JPEG 文件 → 字节序列 → ByteFormer → 数字 0—9

- ① 加载 Apple 官方 ImageNet JPEG 预训练 ByteFormer Tiny。
- ② 保留完整 12 层主干，把原 1000 类分类头换成 10 类。
- ③ 使用 MNIST 继续训练全部参数，输出数字 0—9 的分类结果。

学生只需运行脚本；图像转 JPEG、字节读取、模型加载均已封装。

## 03 MNIST 数据与训练 / 验证 / 测试划分



28×28 灰度图； 10 个数字类别。  
原始文件已随 GitHub 仓库提供。

**训练**  
50,000 张

更新模型参数

**验证**  
1,000 张

每轮检查，选择 best.pt

**测试**  
10,000 张

模型确定后最终评估

官方 60,000 张训练图先分为 50,000 训练池 +10,000 验证池；本课固定取 1,000 验证图，余 9,000 张未使用。

## 04 平台与配套材料：优先 Kaggle，备用 AutoDL

免费额度、可用 GPU 与租用价格，以平台当前页面为准。

| 路线            | 学生要准备什么               | 使用方式                      |
|---------------|-----------------------|---------------------------|
| Kaggle 免费 GPU | 账号、可用 GPU 额度、Internet | 导入课程 Notebook，逐格运行        |
| AutoDL 备用     | 按当前报价租 1 张 GPU        | 选择 PyTorch 镜像，打开 Terminal |
| 本地 CPU        | 已有 Python/PyTorch 环境  | 小样本流程检查，不替代正式实验           |

[Kaggle: 创建 Notebook](#)

[AutoDL: 实例与价格](#)

配套：course\_kaggle.ipynb 逐步操作 | README 完整命令 | Word 实验报告模板。

## 05 Kaggle 准备：导入、GPU、网络，一次设好

1. 从 GitHub 下载 `course_kaggle.ipynb`。
2. 新建 Notebook，File → Import Notebook。
3. 在设置中选择 GPU Accelerator。
4. 打开 Internet，按提示完成账号验证。
5. 从上到下逐格运行，先完成环境检查。

### 成功标志

**GPU available: True**

随后运行“获取课程”单元格。

### 命令规则

Kaggle 单元格： `!python ...`

切换目录： `%cd ...`

AutoDL 终端： `python ...`

按顺序逐格运行：训练单元格会提示输入自己选择的轮数。

## 06 获取代码、数据与预训练权重

Kaggle 笔记本已写好这些命令，按顺序运行即可。

```
!git clone https://github.com/Franklin-L/byteformer-mnist-course.git
%cd /kaggle/working/byteformer-mnist-course
!python -m pip install -r requirements.txt
!python prepare.py
```

看到 [READY] 即准备完成：MNIST 约 11.6MB 随仓库提供；官方权重约 64MB 自动下载并校验。

prepare.py : 准备资源    train.py : 微调    evaluate.py : 评估    predict.py : 预测  
data/ : 数据    checkpoints/ : 预训练权重    outputs/ : 你自己的实验结果

## 07 正式训练：看懂参数，运行一条命令

|            |                      |
|------------|----------------------|
| 训练样本       | 50,000               |
| 验证 / 测试    | 1,000 / 10,000       |
| 训练轮数       | 自行设置                 |
| batch size | 32                   |
| 学习率        | 主干 $1e-4$ ； 分类头 10 倍 |

```
EPOCHS = int(input("epochs: "))  
!python train.py --epochs {EPOCHS}
```

出现 [TRAIN] 和每轮 [EPOCH] ，  
最后看到 [DONE] 。

输出保存到 outputs/baseline/ 。  
训练中不要关闭运行会话。

轮数根据训练与验证表现自行调整；学习率策略可配置，测试集只作最终评估。

## 08 正式实测结果：先用验证选模型，再做测试

最佳验证准确率

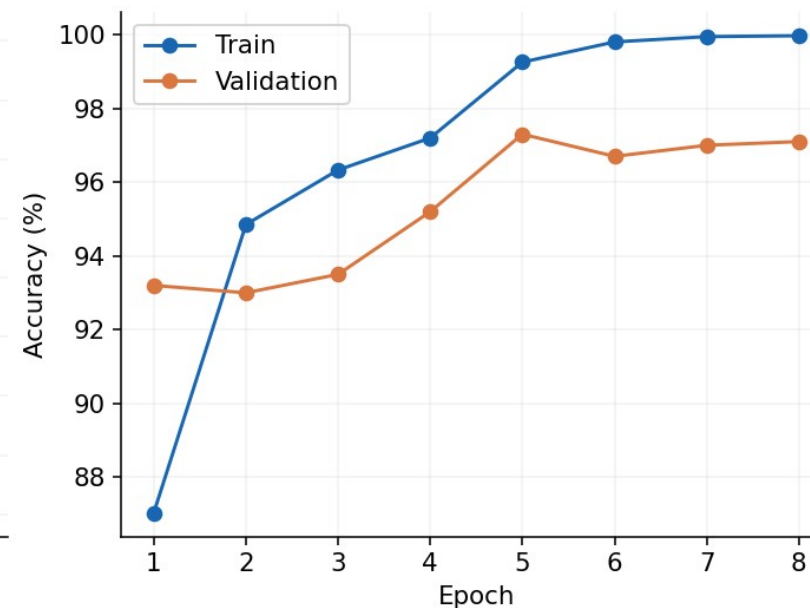
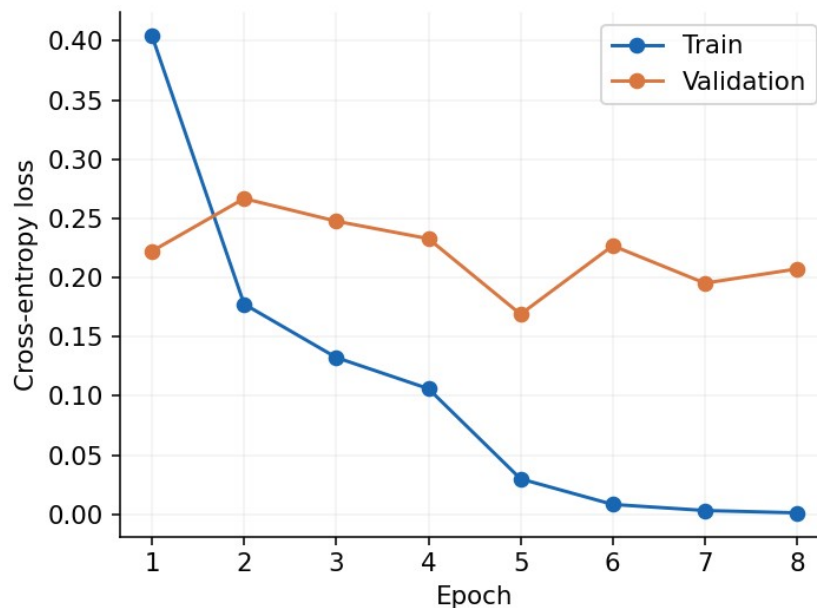
**97.30%**

完整测试准确率

**97.07%**

模型选择依据

**验证集表现**



本次教师实测约 11.88 分钟，不含安装下载及末尾出图。云平台耗时以实际运行为准。

## 09 评估、单图预测与错例检查

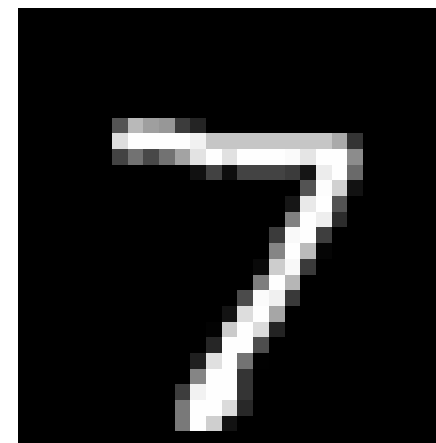
```
!python evaluate.py --checkpoint outputs/baseline/best.pt  
!python predict.py --checkpoint outputs/baseline/best.pt --index 0
```

评估：读取 best.pt ，保存 evaluation.json 。

预测：显示真实标签与预测，生成单图结果。

实操：换一个 0—9999 的测试索引。

错例：Notebook 自动给出错误样本索引。



真实标签 7 | 模型预测 7

Notebook 另提供完整预测图和混淆矩阵，供查找与分析错例。

## 10 学生实操：自行设置，观察验证结果

```
COMPARISON_EPOCHS = int(input(" 对比轮数: "))  
!python train.py --epochs {COMPARISON_EPOCHS} --output outputs/comparison
```

保持样本数、随机种子、batch 和学习率策略不变。

两次都从同一官方预训练权重开始。

保留 baseline，另存 comparison。

用自己的结果回答

- 验证准确率是否提高？
- 训练损失怎样变化？
- 改变设置的收益与代价？
- 为什么还需要验证集？

不固定训练轮数，也不预设训练越久越好。记录自己的设置，并用验证结果解释选择。

# 11 AutoDL 备用路线：开机 → 运行 → 下载 → 关机

租用 1 张 GPU 并选择 PyTorch 镜像 → 打开 JupyterLab → Terminal；以下命令前不加！。

```
cd /root/autodl-tmp
git clone https://github.com/Franklin-L/byteformer-mnist-course.git
cd byteformer-mnist-course
python -m pip install -r requirements.txt
python prepare.py
read -p "epochs: " EPOCHS
python train.py --epochs "$EPOCHS"
```

运行后下载 outputs 中的指标和图；如需继续预测，保存 best.pt。

结束后回控制台关机：关闭浏览器不等于关机。价格以页面当前报价为准。

[AutoDL官方快速开始（实例、JupyterLab、Terminal）](#)

## 12 常见问题：从错误最后一行定位

| 现象                    | 处理                                        |
|-----------------------|-------------------------------------------|
| GPU available: False  | 开启 Kaggle GPU ； 或确认 AutoDL 实例和 PyTorch 环境 |
| No module named ...   | 在当前课程目录安装 requirements.txt ， 确认使用同一环境     |
| 下载超时 / 校验失败           | 开启 Internet 重试；或使用教师预下载资源包                |
| CUDA out of memory    | batch-size 改 16 或 8 ， 两组对比保持一致            |
| 输出目录已存在 / best.pt 找不到 | 重跑使用新 output ； 评估 checkpoint 对应实际训练目录     |
| 准确率低于参考               | 核对样本量、轮数、预训练来源与学习率策略，记录真实结果               |

CPU 备用： README 提供 1,000 张训练图、 1 轮的流程检查命令； 它不代表正式 95% 以上的课程效果。

## 13 下载结果、填写报告并提交

- ① 运行 Notebook 最后的打包单元格。
- ② 在文件面板下载  
byteformer\_mnist\_results.zip。
- ③ 用 Word 模板填写自己的环境、命令、指标和解释。
- ④ 附两组曲线、预测图、错例证据。

默认不用提交数据集、Python 环境或大模型。

### 建议评分 / 100 分

|         |    |
|---------|----|
| 环境与数据   | 15 |
| 基线完成    | 25 |
| 曲线与指标解读 | 20 |
| 单参数对比   | 20 |
| 错例分析    | 10 |
| 报告完整性   | 10 |

提交渠道与截止时间由任课教师说明；只填写真实运行结果，验证与测试指标不要混写。

# 14 课程入口与参考资料

---

详细点击步骤、完整命令和排错说明，统一放在配套指南与 Notebook。

[课程仓库：代码、Notebook、数据、PPT、报告模板](#)

[Kaggle Notebook 官方文档](#)

[AutoDL 官方快速开始](#)

[ByteFormer 原论文：Bytes Are All You Need](#)

[Apple CoreNet：ByteFormer代码与预训练权重](#)

[MNIST：CVDF镜像与原作者说明](#)

校园、校徽来自教师样例并保留原权利；代码、数据与模型归属见仓库说明。